

Тестовое задание - Project Technical Lead

Время: до 8 часов

Формат: MD документ с JSON-схемами, псевдокодом и диаграммами. Если хотите проиллюстрировать логику кодом на TypeScript - приветствуется, но не обязательно.

Срок: 7 дней с момента получения

Контекст:

Мы строим web-based 3D конфигуратор ванной комнаты. Пользователь выбирает продукты из каталога (смесители, душевые стойки, раковины и т.д.), размещает их в комнате и выбирает финиши (хром, матовый чёрный, золото и др.).

У клиента есть собственная система правил совместимости продуктов и финишей. Формат и стек этой системы нам неизвестны - это может быть Excel, REST API, или своя база данных. Правила могут быть такими:

- Смеситель серии A совместим только с раковинами серии B и C
- Финиш «золото» недоступен для душевых стоек бюджетной коллекции
- Если выбран продукт X, продукт Y автоматически становится недоступен
- Некоторые комбинации требуют обязательного добавления третьего продукта

Для понимания задачи - базовая структура каталога:

- **Коллекция** - линейка продуктов одного производителя (например «Коллекция Loft»). Объединяет продукты по стилю.
- **Категория** - тип продукта: смеситель для раковины, душевая стойка, полотенцесушитель, раковина и т.д. Около 8 - 12 категорий.
- **Продукт** - конкретная позиция в каталоге с SKU, названием, ценой и набором допустимых финишей.
- **Финиш** - вариант поверхностной отделки: хром, матовый чёрный, золото и др. Один финиш может применяться к продуктам из разных коллекций и категорий.
- **Правила совместимости** задаются на уровне коллекций, категорий, или конкретных продуктов - это и нужно спроектировать.

Масштаб: десятки коллекций, сотни продуктов, единицы финишей (5-10).

Задача 1 - Rules Engine

Спроектируйте внутреннюю архитектуру rules engine на нашей стороне, независимо от того в каком формате приходят правила от клиента.

Ожидаем:

- Схему данных: как правила хранятся внутри нашей системы
- Типологию правил: какие типы правил существуют, как они выражаются универсально
- Механизм применения: как engine проверяет совместимость при выборе пользователем продукта или финиша в реальном времени
- Адаптерный слой: как правила от клиента (любой формат) нормализуются в нашу внутреннюю схему
- Обработка конфликтов: что происходит когда правила противоречат друг другу

Задача 2 - Система материалов

Спроектируйте архитектуру системы финишей конфигуратора.

Ожидаем:

- Схему данных каталога финишей: как хранятся PBR параметры, текстуры, метаданные
- Конвенцию именования материальных слотов в GLB моделях: как художник и разработчик работают по единому стандарту
- Механизм применения: как финиш применяется на все продукты нужного типа в сцене одновременно
- Расширяемость: добавление нового финиша должно требовать только изменения конфига, не кода - покажите как это достигается
- Связь с rules engine: как ограничения на финиши из задачи 1 интегрируются с этой системой